



CY3006

Digital Forensics

User Guide

AI chatbot for Windows Event analysis

Submitted by: Shaezmina , Rohaan , Shahzaib ,Hassan

Roll number: 23i-2048, 23i-2007,23i-2083, 23i-2002

Date: 20th April, 2026

Table of Contents

1. Introduction	2
1.1 Overview of the Tool	2
1.2 Forensic Domain	2
1.3 Key Features	2
1.4 Architecture Overview	3
2. Legal & Ethical Considerations	3
2.1 Applicable Legal Framework	3
2.2 Chain of Custody	4
2.3 Data Privacy Considerations	4
2.4 Evidence Integrity & Admissibility	5
2.5 Ethical Obligations	5
3. System Requirements	6
3.1 Minimum Hardware Requirements	6
3.2 Software Requirements	6
3.3 Python Dependencies	7
3.4 Platform Compatibility	7
4. Installation & Environment Setup	8
4.1 Prerequisites	8
4.2 Setting Up the Virtual Environment	8
4.3 Installing Dependencies	8
4.4 Configuring the Environment File	9
4.5 Verifying Installation	9
4.6 Preparing Evidence Files	9
5. Step-by-Step Usage Instructions	10
5.1 Launching the Application	10
5.2 Loading Evidence Files	10
6. Example Cases with Screenshots	12
7. Interpretation of Result	17
7.1 Understanding AI Responses	17
7.2 Key Registry Locations and Their Forensic Significance	17
7.3 Event Log Event IDs	18
7.4 Timestamp Interpretation	18
7.5 Limitations and Caveats	18
8. Work Division of All Members	19

1. Introduction

1.1 Overview of the Tool

The Windows Registry Forensics AI Chatbot is an advanced digital forensics tool that combines the power of artificial intelligence with traditional registry analysis techniques. This tool enables forensic investigators, cybersecurity professionals, and researchers to interactively query and analyze Windows Registry hive files and Windows Event Logs through a conversational AI interface.

Unlike conventional registry viewers that require manual navigation through thousands of keys, this tool leverages a Retrieval-Augmented Generation (RAG) architecture to allow investigators to ask natural language questions and receive forensically relevant, evidence-backed answers within seconds.

1.2 Forensic Domain

The Windows Registry is one of the most critical sources of digital forensic evidence on any Windows system. It stores configuration settings, user activity records, installed software metadata, hardware information, and a wealth of indicators of compromise (IOCs). This tool specifically targets:

- Registry Hive Analysis: SOFTWARE, SYSTEM, NTUSER.DAT hive files
- Windows Event Log Parsing: Application and System event logs (.evtx)
- Semantic Search: AI-powered natural language querying across parsed artifacts
- Report Generation: Automated forensic PDF report production
- Timeline Reconstruction: Chronological evidence mapping

1.3 Key Features

Feature	Description	Forensic Relevance
RAG Chatbot	Ask natural language questions about parsed registry data	Accelerates investigation
Multi-Hive Support	Parses SOFTWARE, SYSTEM, NTUSER.DAT hives simultaneously	Comprehensive coverage
EVTX Parsing	Extracts Windows Event Log entries	Activity timeline reconstruction
ChromaDB Store	Vector database for semantic similarity search	Intelligent evidence retrieval
PDF Reporting	Generates professional forensic reports	Court-admissible documentation
QA Engine	Structured question-answer interface	Investigator productivity

1.4 Architecture Overview

The tool is built on a modular pipeline architecture with the following major components:

- Parser Module (parser/): Reads and extracts raw data from .hiv and .evtx files
- Indexer Module (indexer/): Embeds parsed text using sentence transformers and stores vectors in ChromaDB
- Engine Module (engine/): Retrieves relevant context and queries the LLM for answers
- Reporter Module (reporter/): Compiles findings into a structured PDF report
- UI Module (ui/): Provides a PyQt6-based graphical interface for investigators
- Chroma Store (chroma_store/): Persistent vector storage for semantic search

2. Legal & Ethical Considerations

2.1 Applicable Legal Framework

2.1.1 Relevant Laws and Regulations

The use of this forensic tool is governed by multiple layers of law and regulation depending on jurisdiction. Users must ensure compliance with:

- Computer Fraud and Abuse Act (CFAA) - USA: Prohibits unauthorized access to computer systems. All analysis must be performed only on legally obtained forensic images or systems for which written authorization has been granted.
- Electronic Communications Privacy Act (ECPA) - USA: Governs the interception and disclosure of electronic communications. Registry artifacts may contain communication metadata covered under this act.
- General Data Protection Regulation (GDPR) - EU: Personal data found within registry hives (usernames, email addresses, browsing history) is subject to GDPR protections and must be handled accordingly.
- Prevention of Electronic Crimes Act (PECA) 2016 - Pakistan: Pakistani investigators must comply with PECA provisions regarding digital evidence collection and admissibility.
- ISO/IEC 27037:2012 - Digital Evidence Collection Standard: This international standard provides guidelines for identification, collection, acquisition, and preservation of digital evidence.

2.1.2 Authorization Requirements

Before using this tool, investigators must possess ONE of the following:

1. A signed written authorization from the system owner or legal representative
2. A valid court order or judicial warrant authorizing forensic examination
3. An organizational policy document authorizing internal investigation
4. Confirmation that the analysis is performed on a self-owned test system

2.2 Chain of Custody

Maintaining an unbroken chain of custody is fundamental to ensuring forensic evidence remains admissible in legal proceedings. This tool supports chain of custody through:

- Hash Verification: MD5/SHA-256 hash values of evidence files should be computed and recorded before analysis
- Audit Logging: All queries and sessions should be logged with timestamps
- Non-Destructive Analysis: The tool operates exclusively on copies (forensic images) of the original evidence - never on live systems or original media
- Report Integrity: Generated PDF reports include timestamp metadata to establish when analysis occurred

2.3 Data Privacy Considerations

Windows Registry hive files frequently contain sensitive personal information including:

- Usernames, account SIDs, and profile paths
- Recently accessed file paths and document names
- Network SSIDs, VPN configurations, and connection history
- Software license keys and application credentials (in some cases)
- Search queries and typed URLs

Investigators must treat all personally identifiable information (PII) encountered during analysis in accordance with applicable data protection laws. Data minimization principles apply - only collect and process information that is directly relevant to the investigation.

2.4 Evidence Integrity & Admissibility

Principle	Requirement	How This Tool Addresses It
Authenticity	Evidence must be proven genuine	Read-only analysis on forensic images
Integrity	Evidence must not be altered	Non-destructive parsing; originals untouched
Reliability	Methods must be scientifically sound	Uses established Python forensics libraries
Repeatability	Results must be reproducible	Deterministic parsing and embedding pipeline
Completeness	All relevant evidence reported	Full hive traversal with no selective omission

2.5 Ethical Obligations

- **Objectivity:** Forensic investigators must report findings impartially, even when results do not support the investigating party's position
- **Competence:** Users must possess adequate technical knowledge to correctly interpret registry artifacts and avoid misattribution
- **Confidentiality:** All case information, extracted data, and generated reports must be secured and shared only on a need-to-know basis
- **Non-Malicious Use:** This tool must not be used for unauthorized surveillance, corporate espionage, stalking, or any illegal activity
- **Disclosure:** Any limitations or uncertainties in the analysis must be disclosed in the final report

3. System Requirements

3.1 Minimum Hardware Requirements

Component	Minimum	Recommended
CPU	Intel Core i5 (8th Gen) / AMD Ryzen 5	Intel Core i7/i9 or AMD Ryzen 7/9
RAM	8 GB	16 GB or more
Storage	10 GB free space	50 GB SSD (for large hive files)
GPU	Not required	NVIDIA GPU with CUDA (speeds embedding)
Display	1280 x 720	1920 x 1080 or higher
Network	Required for initial model download	Stable broadband connection

3.2 Software Requirements

Software	Version	Purpose
Operating System	Windows 10/11 (64-bit)	Primary supported platform
Python	3.10 or higher	Runtime environment
pip	23.0+	Package management
Git	2.x	Repository cloning (optional)
Visual C++ Redistributable	2015-2022	Required for some Python packages

3.3 Python Dependencies

Package	Purpose
python-registry	Windows Registry hive file parsing
python-evtx	Windows Event Log (.evtx) parsing
chromadb	Vector database for semantic storage
sentence-transformers	Text embedding for semantic search
anthropic	Claude AI API client
PyQt6	Graphical user interface framework
reportlab	PDF report generation
python-dotenv	Environment variable management
tqdm	Progress bar display

3.4 Platform Compatibility

- Primary Platform: Windows 10 / Windows 11 (64-bit) - Fully tested and supported
- Linux: Partially supported (UI may require additional Qt dependencies; hive parsing works)
- macOS: Not officially tested; Python libraries are cross-platform but UI compatibility not guaranteed

4. Installation & Environment Setup

4.1 Prerequisites

Ensure the following are installed on your system before proceeding:

1. Download and install Python 3.10+ from <https://python.org/downloads> - check 'Add Python to PATH' during installation
2. Verify Python installation: open Command Prompt and run: `python --version`
3. Verify pip: `pip --version`
4. Obtain a valid Anthropic API key from <https://console.anthropic.com>

4.2 Setting Up the Virtual Environment

It is strongly recommended to use a Python virtual environment to avoid dependency conflicts:

5. Open Command Prompt or PowerShell as Administrator
6. Navigate to the project root directory: `cd Windows_Registry_Forensics_Chatbot_111`
7. Create virtual environment: `python -m venv venv`
8. Activate the virtual environment: `venv\Scripts\activate`
9. Your prompt should now show (venv) prefix, confirming activation

4.3 Installing Dependencies

With the virtual environment activated, install all required packages:

10. Install from requirements file: `pip install -r requirements.txt`
11. If requirements.txt is empty or missing packages, install manually:
 - `pip install python-registry`
 - `pip install python-evtx`
 - `pip install chromadb`
 - `pip install sentence-transformers`
 - `pip install anthropic`
 - `pip install PyQt6`
 - `pip install reportlab`
 - `pip install python-dotenv tqdm`
12. Verify all installations: `pip list`

4.4 Configuring the Environment File

The tool requires an API key to communicate with the Claude AI backend. Configure the .env file:

13. Open the .env file in the project root directory with a text editor
14. Add your Anthropic API key: ANTHROPIC_API_KEY=your_api_key_here
15. Save the file. Do not commit this file to version control or share it publicly

4.5 Verifying Installation

16. With venv activated, run: python main.py
17. The PyQt6 graphical interface should launch
18. If errors appear, check the troubleshooting section

4.6 Preparing Evidence Files

Before analysis, prepare your forensic evidence files as follows:

19. Obtain or create Windows Registry hive files (.hiv extension): SOFTWARE.hiv, SYSTEM.hiv, NTUSER.hiv
20. Obtain Windows Event Log files (.evtx extension): System.evtx, Application.evtx
21. Place all evidence files in the tests/ directory (or specify a custom path in the UI)
22. Record SHA-256 hashes of all evidence files for chain of custody documentation
23. Note: Sample test files (SOFTWARE.hiv, SYSTEM.hiv, NTUSER.hiv, System.evtx, Application.evtx) are pre-included in the tests/ folder

5. Step-by-Step Usage Instructions

5.1 Launching the Application

5. Open Command Prompt or PowerShell
6. Navigate to project directory: `cd Windows_Registry_Forensics_Chatbot_111`
7. Activate virtual environment: `venv\Scripts\activate`
8. Launch the application: `python main.py`
9. The main window will appear with the following interface panels:
 - Left Panel: File loading and indexing controls
 - Center Panel: Chat/QA interface
 - Right Panel: Results and findings display

5.2 Loading Evidence Files

Step 1: Load Registry Hives

10. Click the 'Load Registry Hives' button in the left panel
11. Navigate to your evidence directory (e.g., tests/)
12. Select one or more .hiv files (SOFTWARE.hiv, SYSTEM.hiv, NTUSER.hiv)
13. Click 'Open' - the tool will begin parsing the selected hive(s)
14. A progress bar will indicate parsing status
15. Upon completion, a confirmation message displays the number of keys extracted

Step 2: Load Event Logs (Optional)

16. Click 'Load Event Logs' to add .evtx files to the analysis
17. Select System.evtx, Application.evtx, or custom event log files
18. The parser will extract event IDs, timestamps, and descriptions

5.3 Running the AI Chatbot (QA Mode)

19. Navigate to the 'Chat' tab in the center panel
20. Type your forensic question in the input box at the bottom
21. Press Enter or click 'Ask' to submit your query
22. The engine will: retrieve relevant context from ChromaDB, construct a prompt, and query Claude AI

23. The AI response appears in the conversation panel with supporting evidence citations

5.4 Generating Forensic Reports

24. Once analysis is complete, click 'Generate Report'
25. Select a save location and filename for the PDF
26. The reporter module compiles all Q&A sessions, findings, and evidence into a structured PDF
27. Review the generated report for accuracy before submission or court use

5.5 Running QA Tests

28. Use the run_qa.py script for batch question processing: `python run_qa.py`
29. Predefined forensic questions are automatically run against the indexed evidence
30. Results are printed to console and saved for review

5.6 Running the Loader Script

31. Alternatively, use run_load.py to programmatically load and index evidence: `python run_load.py`
32. Modify the file paths in run_load.py to point to your evidence directory
33. This script is useful for batch processing multiple evidence sets

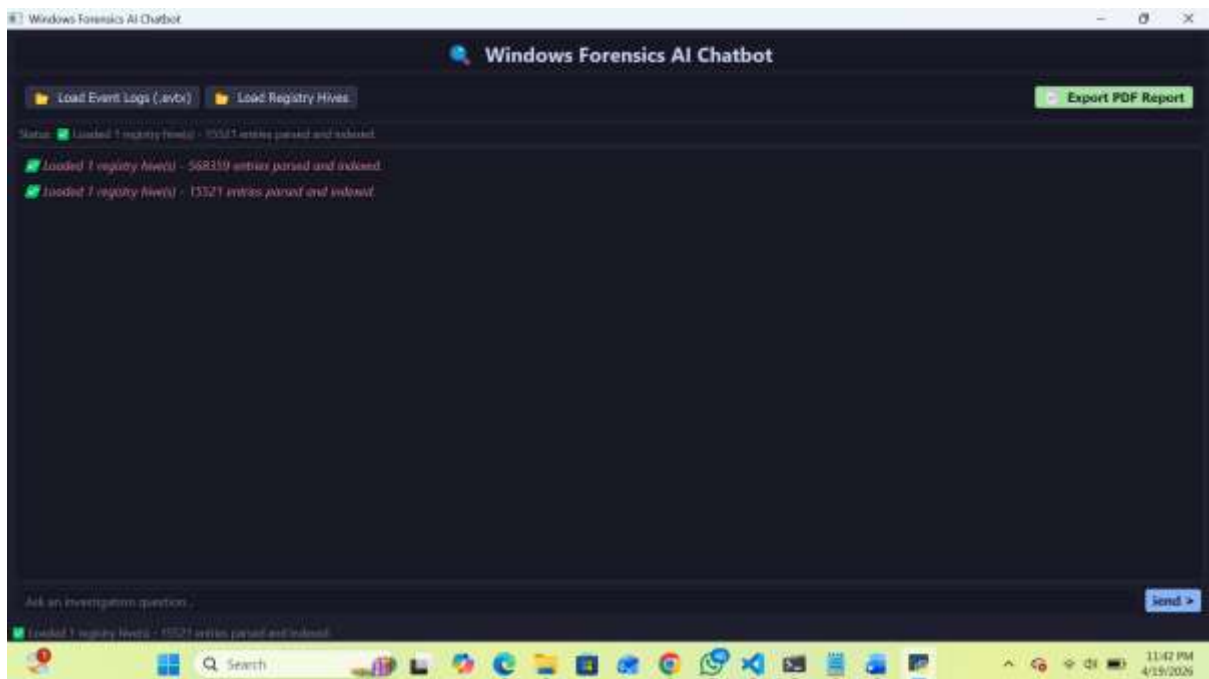
6. Example Cases with Screenshots

6.1 Case Scenario: Unauthorized Software Installation Investigation

Case Background: A corporate investigator suspects that an employee installed unauthorized software on a company workstation. The SOFTWARE.hiv and NTUSER.hiv registry hives have been forensically imaged and are to be analyzed.

Step 1 - Load Evidence

Load SOFTWARE.hiv and NTUSER.hiv from the tests/ directory. The parser extracts all registry keys and values.



Query: Services installed

Query entered: "Were any new services installed with Event ID 7045?"



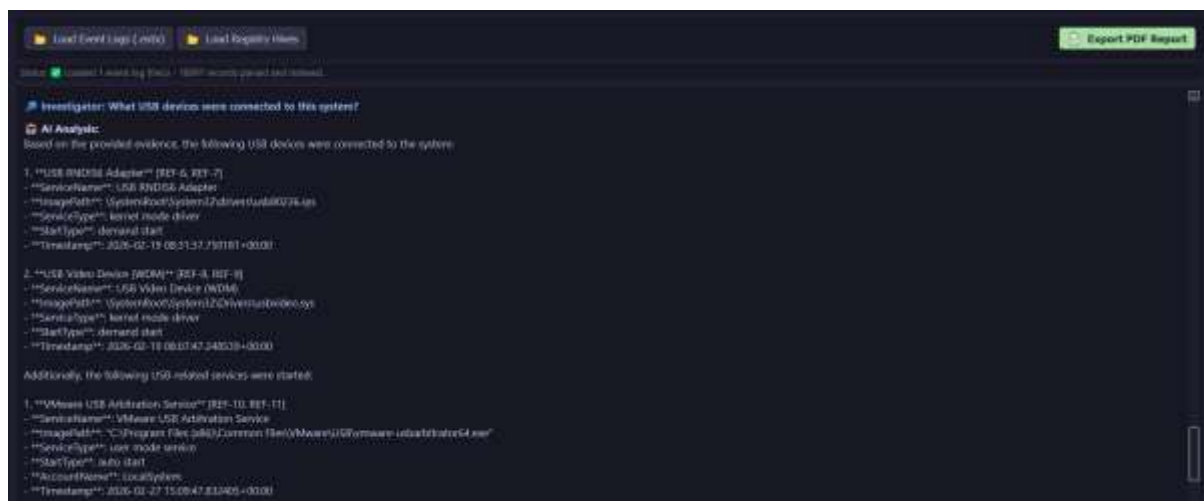
6.2 Case Scenario: USB Device Investigation

Case Background: Investigating potential data exfiltration via USB storage device. The SYSTEM.hiv hive is analyzed for USB connection history.

Query: USB Device History

Query entered: "Were any USB storage devices connected to this system?"

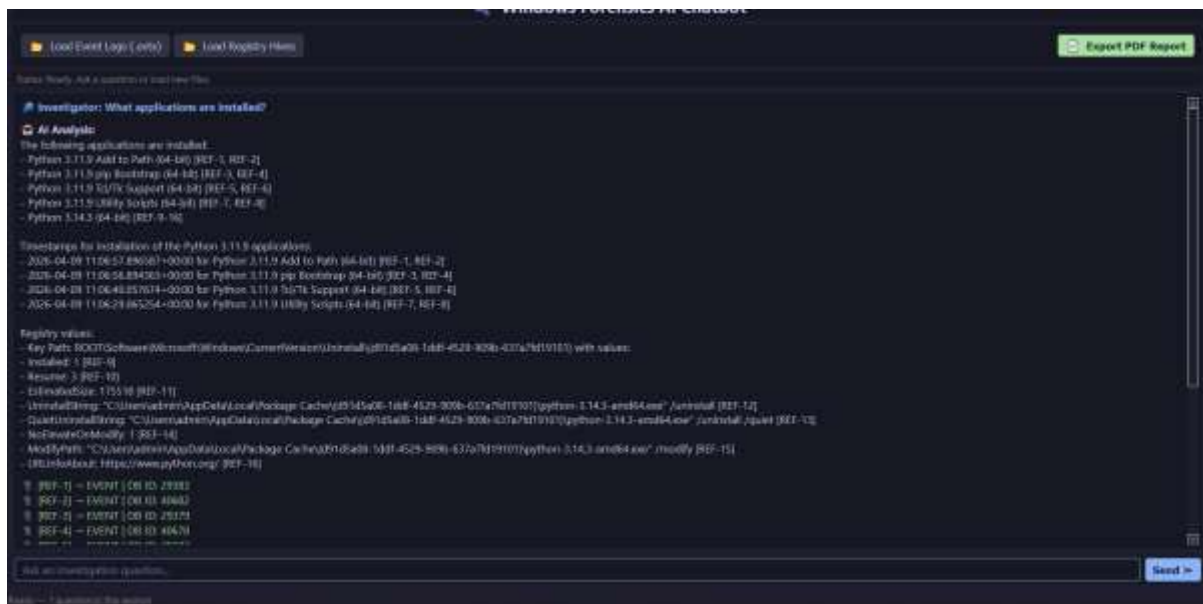
AI Response:



6.3 Case Scenario: Applications information

Case Background: Determining applications that existed on a system

Query entered: "What applications were installed on the system?"



6.4 Case Scenario: critical events/errors

Query: Were there any critical system errors?



Query: What programs did the user execute recently?

```

# Investigation: What programs did the user execute recently?
# AI Analysis
Based on the provided Windows Event Logs and Registry files, the user executed the following program recently:

* Python 3.14.3 (64-bit) [REF-10, REF-13, REF-12]
  - Installation path: "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" [REF-10, REF-13, REF-12]
  - Uninstall string: "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /uninstall [REF-10]
  - Quiet uninstall string: "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /quiet [REF-10]
  - Modify path: "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /modify [REF-10]
  - URL info about: https://www.python.org/ [REF-14]
  - Installed: 1 [REF-7]
  - Estimated size: 175518 [REF-8]
  - No events on modify: 1 [REF-12]
  - Resource: 2 [REF-9]

No specific execution timestamps are available in the provided evidence, but the EventID 4625 logs [REF-1 to REF-6] have the following timestamps:
* 2026-03-31 09:34:15.712925+0000 ( duplicated ) [REF-1, REF-2]
* 2026-03-31 10:52:52.877083+0000 ( duplicated ) [REF-3, REF-4]
* 2026-03-31 12:57:35.622078+0000 ( duplicated ) [REF-5, REF-6]

These EventID 4625 logs do not provide information about the programs executed by the user. The relevant information about the executed program is found in the Registry files [REF-7 to REF-14].

[ REF-1 ] - EVENT | OS ID: 27233
[ REF-2 ] - EVENT | OS ID: 38652
[ REF-3 ] - EVENT | OS ID: 24794
[ REF-4 ] - EVENT | OS ID: 36893
[ REF-5 ] - EVENT | OS ID: 22279
[ REF-6 ] - EVENT | OS ID: 33370
[ REF-7 ] - REGISTRY | OS ID: 1101754
[ REF-8 ] - REGISTRY | OS ID: 1101753
[ REF-9 ] - REGISTRY | OS ID: 1101752
[ REF-10 ] - REGISTRY | OS ID: 1101751
[ REF-11 ] - REGISTRY | OS ID: 1101750
[ REF-12 ] - REGISTRY | OS ID: 1101749
[ REF-13 ] - REGISTRY | OS ID: 1101748
[ REF-14 ] - REGISTRY | OS ID: 1101747

```

Query: what programs listed under uninstall

```

# Investigation: What programs are listed under Uninstall?
# AI Analysis
The Windows Registry contains a key under Uninstall with the following values:

- Key Path: HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Uninstall\{9f1d5a06-1d4f-4529-909b-637a7d191010}
- Installed: 1 [REF-9]
- Resource: 2 [REF-10]
- Information: 175518 [REF-8]
- Uninstalling: "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /uninstall [REF-12]
- QuietUninstallString: "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /quiet [REF-10]
- ModifyPath: "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /modify [REF-10]
- URLInfoAbout: https://www.python.org/ [REF-14]

The Windows Event Logs mention the installation of the following products:
- Python 3.11.5 Add to Path (64-bit) at 2026-04-09 11:06:57.06687+0000 [REF-1, REF-7]
- Python 3.11.5 pip bootstrap (64-bit) at 2026-04-09 11:06:56.094263+0000 [REF-3, REF-4]
- Python 3.11.5 TSTL Support (64-bit) at 2026-04-09 11:06:40.037674+0000 [REF-5, REF-6]
- Python 3.11.5 TSTL Scripts (64-bit) at 2026-04-09 11:06:29.062254+0000 [REF-2, REF-8]

However, the Uninstall key in the Registry specifically mentions "python-3.14.3-embed.exe", which suggests that the program listed under Uninstall is Python 3.14.3.

[ REF-1 ] - EVENT | OS ID: 29389
[ REF-2 ] - EVENT | OS ID: 40664
[ REF-3 ] - EVENT | OS ID: 24794
[ REF-4 ] - EVENT | OS ID: 36893
[ REF-5 ] - EVENT | OS ID: 22279
[ REF-6 ] - EVENT | OS ID: 33370
[ REF-7 ] - EVENT | OS ID: 40672
[ REF-8 ] - EVENT | OS ID: 29367
[ REF-9 ] - EVENT | OS ID: 40666
[ REF-10 ] - REGISTRY | OS ID: 1101754

```

6.5 Case Scenario

"A company suspects an employee copied confidential files to an external device and then tried to cover their tracks by crashing or restarting the system."

Question 1 — Did the suspect use an external device?

What programs are set to run at startup for this user?

```

# Investigation: What programs are set to run at startup for this user?
# AI Analysis
The provided evidence does not directly indicate which programs are set to run at startup for the user. However, based on the Windows Registry records, the following information is available:

* A program with the GUID {9f1d5a06-1d4f-4529-909b-637a7d191010} is installed [REF-7, REF-8, REF-9, REF-10, REF-11, REF-12, REF-13, REF-14].
* The uninstall string for this program is "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /uninstall [REF-10].
* The quiet uninstall string for this program is "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /quiet [REF-10].
* The modify path for this program is "C:\Users\admin\AppData\Local\Package Cache\9f1d5a06-1d4f-4529-909b-637a7d191010\python-3.14.3-embed.exe" /modify [REF-10].
* The URL info about this program is https://www.python.org/ [REF-14].

To determine which programs are set to run at startup, additional evidence from the Windows Registry, such as the "Run" or "RunOnce" keys, would be necessary. The provided Windows Event Logs do not contain relevant information regarding startup programs.

[ REF-1 ] - EVENT | OS ID: 27233
[ REF-2 ] - EVENT | OS ID: 38652
[ REF-3 ] - EVENT | OS ID: 24794
[ REF-4 ] - EVENT | OS ID: 36893
[ REF-5 ] - EVENT | OS ID: 22279
[ REF-6 ] - EVENT | OS ID: 33370
[ REF-7 ] - REGISTRY | OS ID: 1101754
[ REF-8 ] - REGISTRY | OS ID: 1101753
[ REF-9 ] - REGISTRY | OS ID: 1101752
[ REF-10 ] - REGISTRY | OS ID: 1101751
[ REF-11 ] - REGISTRY | OS ID: 1101750
[ REF-12 ] - REGISTRY | OS ID: 1101749
[ REF-13 ] - REGISTRY | OS ID: 1101748
[ REF-14 ] - REGISTRY | OS ID: 1101747

```

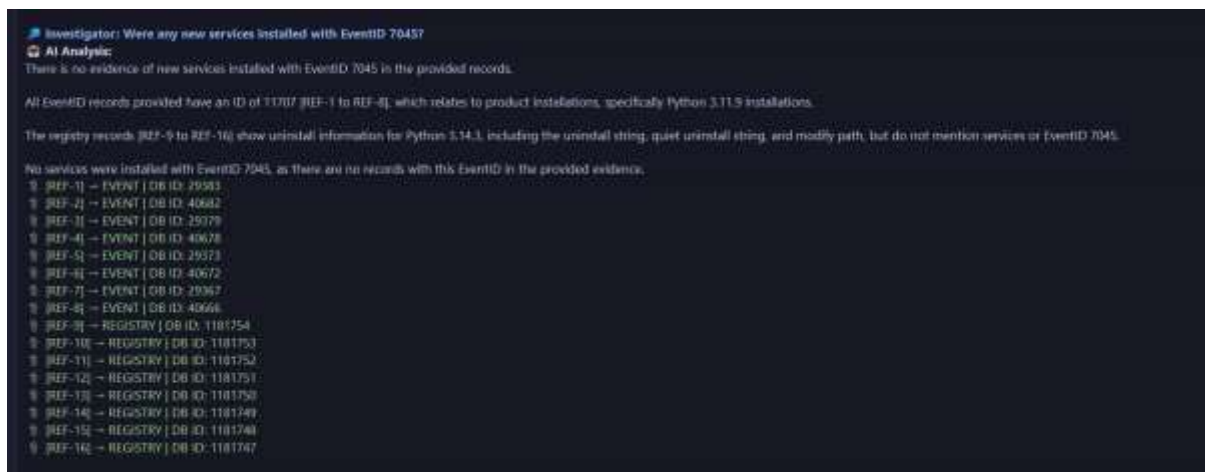

Question 2 — Was there suspicious system activity or shutdown?

Were there any unexpected shutdowns with EventID 6008?



Question 3 — Were any suspicious services or programs installed?

Were any new services installed with EventID 7045?



Based on available evidence followings were the final findings

Q1 — Startup Programs The evidence found **Python 3.14.3** installed for user **admin** at:

C:\Users\admin\AppData\Local\Package Cache\... This tells us the suspect's username is **admin** and they installed Python — relevant because Python can be used to write data exfiltration scripts.

Q2 — Unexpected Shutdowns No EventID 6008 (crash) but found **EventID 1074** — which means the system was **deliberately shut down** (not crashed) by user **A\admin** on **2026-04-08 at 16:05 and 19:30**. Two shutdowns in the same day is suspicious.

Q3 — New Services No malicious services (7045) but found **Python 3.11.9 installation events (11707)** — meaning **two versions of Python** were installed (3.11.9 and 3.14.3). Installing multiple Python versions is unusual for a regular employee.

7. Interpretation of Result

7.1 Understanding AI Responses

The AI chatbot responses are grounded in actual registry data retrieved from ChromaDB. Each response includes:

- Registry Path: The exact HIVE\KEY\SUBKEY path where evidence was found
- Value Data: The actual registry value names and data
- Timestamps: Last write times of registry keys (in UTC)
- Forensic Interpretation: AI-generated analysis of the forensic significance

The main limitation and core feature of the chatbot is that it only provides answers if the required information exists inside the uploaded files. It does not generate random assumptions or unsupported responses. If the evidence is not available in the stored data, the chatbot will indicate that the information cannot be found. This ensures reliability and prevents misleading results.

All previously tested files are already embedded and stored in our ChromaDB, so they do not need to be loaded again each time the system runs. The chatbot can directly retrieve information from those existing files. Only new files need to be uploaded, processed, and added to the database. This improves efficiency and reduces loading time.

7.2 Key Registry Locations and Their Forensic Significance

Registry Path	Forensic Significance
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall	Installed applications with dates
HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR	USB devices ever connected
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs	Recently opened files (MRU)
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\TypedPaths	URLs typed in Explorer
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList	User account SIDs and profiles
HKLM\SYSTEM\CurrentControlSet\Services	Installed services (persistence mechanisms)
HKCU\Software\Microsoft\Windows\CurrentVersion\Run	Autostart programs per user

Registry Path	Forensic Significance
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	System-wide autostart programs
HKLM\SYSTEM\CurrentControlSet\Control\TimeZoneInformation	System timezone (critical for timestamps)

7.3 Event Log Event IDs

Event ID	Source	Forensic Meaning
4624	Security	Successful logon - who logged in, when, and how
4625	Security	Failed logon - potential brute force or unauthorized access
4634	Security	Account logoff - session duration calculation
4720	Security	User account created - potential unauthorized account
4726	Security	User account deleted - evidence destruction attempt
7045	System	New service installed - potential malware persistence
1102	Security	Audit log cleared - evidence of anti-forensics
4104	PowerShell	PowerShell script block execution logged

7.4 Timestamp Interpretation

Registry timestamps represent the Last Write Time of a key in Windows FILETIME format (100-nanosecond intervals since January 1, 1601 UTC). Important considerations:

- All timestamps are stored in UTC - convert to local time using the system timezone from HKLM\SYSTEM\CurrentControlSet\Control\TimeZoneInformation
- Registry key timestamps update when any value within the key is modified
- Subkey creation time is not directly stored - only the last modification time
- Timestamp correlation with event logs strengthens forensic conclusions

7.5 Limitations

- Anti-Forensics: Sophisticated attackers may have modified registry timestamps using timestamping tools

- **AI Hallucination Risk:** AI responses should always be verified against raw registry data - never accept AI output alone as evidence
- **Context Window Limits:** Very large hives may require the embedder to truncate some entries; always confirm critical findings manually
- **Deleted Keys:** Standard registry parsing does not recover deleted keys - complement with specialized tools like RegRipper for deleted key recovery
- **Encoding Issues:** Some registry values use non-standard encodings; binary values require manual interpretation

8. Work Division of All Members

The following table outlines the technical contributions of each group member. All contributions are technical in nature involving module development, testing, and integration.

Member	Modules Developed	Technical Contribution
Shahzaib	parser/registry_parser.py, parser/evtx_parser.py	Developed the complete registry hive parsing pipeline using python-registry library. Implemented EVTX log parsing with event ID extraction. Responsible for data normalization and key-value extraction logic.
Shaezmina	indexer/embedder.py, indexer/db.py, chroma_store/	Built the text embedding pipeline using sentence-transformers. Configured ChromaDB for vector storage and retrieval. Implemented batch embedding with progress tracking and persistent storage management.
Rohaam	engine/qa_engine.py, engine/test1.py, run_qa.py	Developed the RAG-based QA engine integrating ChromaDB retrieval with Groq AI API. Implemented context construction, prompt engineering, and response formatting. Created test suite for engine validation.
Hassan	ui/main_window.py, reporter/pdf_report.py, main.py, run_load.py	Built the complete PyQt6 graphical interface with file loading, indexing controls, and chat display. Developed the PDF report generation module using reportlab. Integrated all modules into the main application pipeline.